Giovanna Paiva Alves

FakeNumDetect: Biblioteca Python para Detecção de Anomalias em Dados Numéricos Fabricados usando Métodos Estatísticos e Machine Learning

São Paulo
2026

Giovanna Paiva Alves

FakeNumDetect: Biblioteca Python para Detecção de Anomalias em Dados Numéricos Fabricados usando Métodos Estatísticos e Machine Learning

Proposta de Trabalho de Conclusão de Curso apresentado ao SENAC Santo Amaro como requisito parcial para aprovação na disciplina de TCC1 do curso de Bacharelado em Ciência da Computação.

SENAC – Serviço Nacional de Aprendizagem Comercial
Unidade Santo Amaro
Curso de Bacharelado em Ciência da Computação

Orientador: Prof. Afonso César Lelis Brandão

São Paulo
2026

Resumo

Este documento apresenta a proposta inicial do Trabalho de Conclusão de Curso (TCC1) desenvolvido no SENAC Santo Amaro. O objetivo deste trabalho é [descrever brevemente o objetivo principal do TCC]. A justificativa para este estudo baseia-se em [descrever brevemente a justificativa]. A revisão bibliográfica abrange [mencionar os principais tópicos]. Como solução proposta, pretende-se [descrever brevemente a solução]. Este trabalho está organizado em três capítulos que apresentam a introdução ao tema, a revisão bibliográfica que fundamenta a pesquisa, o desenvolvimento com a solução proposta e o cronograma de trabalho, seguidos das referências bibliográficas.

Palavras-chave: palavra1. palavra2. palavra3. palavra4. palavra5.

Sumário

1 INTRODUÇÃO . . . 4
1.1 Contexto . . . 4
1.2 Justificativa . . . 5
1.3 Objetivo . . . 5
1.3.1 Objetivo principal . . . 5
1.3.2 Objetivos secundários . . . 6

2 REVISÃO BIBLIOGRÁFICA . . . 7
2.1 Referencial teórico . . . 7
2.1.1 Forense numérica e integridade de dados . . . 7
2.1.2 Lei de Benford . . . 7
2.1.3 Teste GRIM . . . 8
2.1.4 Teste SPRITE . . . 8
2.1.5 Análise de entropia de Shannon . . . 9
2.1.6 Isolation Forest . . . 9
2.1.7 Autoencoders para detecção de anomalias . . . 9
2.2 Metodologia da pesquisa bibliográfica . . . 10
2.3 Fundamentos técnicos . . . 10
2.3.1 Design de API estilo scikit-learn . . . 10
2.3.2 Score de confiança unificado . . . 11
2.3.3 Geração de relatórios forenses . . . 11
2.4 Trabalhos relacionados . . . 11
2.4.1 Trabalho 1: *Tools of the data detective* (Crone e Green, 2025) . . . 11
2.4.2 Trabalho 2: *Integrating Machine Learning with Digital Forensics* (Ndibe, 2025) 12
2.4.3 Trabalho 3: *Integrating Data Mining Techniques for Fraud Detection* (Sushkov et al., 2023) . . . 12

3 DESENVOLVIMENTO . . . 14
3.1 Solução proposta . . . 14
3.2 Cronograma de trabalho . . . 14

REFERÊNCIAS . . . 16

1 Introdução

A fabricação e manipulação de dados numéricos representa um problema crescente em múltiplos domínios, desde fraudes fiscais e contábeis até a falsificação de resultados em pesquisas científicas. Este trabalho propõe o desenvolvimento da **FakeNumDetect**, uma biblioteca open-source em Python que unifica diversas técnicas estatísticas e de aprendizado de máquina para detecção de anomalias em dados numéricos potencialmente fabricados.

1.1 Contexto

A integridade de dados numéricos é um requisito fundamental em domínios como auditoria fiscal, pesquisa científica e jornalismo de dados. A fabricação e a manipulação de valores numéricos são práticas documentadas e recorrentes: no campo científico, estudos estimam que entre 2% e 10% das publicações contêm dados fabricados ou significativamente manipulados (SIMONSOHN, 2013), comprometendo a reprodutibilidade do conhecimento produzido. No âmbito financeiro e fiscal, a adulteração de valores em notas fiscais, balanços e contratos gera perdas estimadas em cerca de 5% da receita anual das organizações (USTINOVA, 2022). Ambos os cenários exigem ferramentas capazes de detectar anomalias numéricas de forma sistemática e auditável. Para enfrentar esse problema, a literatura consolidou um conjunto de técnicas de forense numérica. A Lei de Benford (BENFORD, 1938) estabelece que, em conjuntos de dados gerados por processos naturais, o primeiro dígito significativo segue uma distribuição logarítmica previsível — desvios dessa distribuição sinalizam possível manipulação. Os testes GRIM (*Granularity-Related Inconsistency of Means*) e SPRITE (*Sample Parameter Reconstruction via Iterative Techniques*) verificam se as estatísticas descritivas reportadas em artigos científicos são matematicamente compatíveis com o tamanho e a granularidade da amostra declarada (BROWN; HEATHERS, 2017; HEATHERS et al., 2018). Complementarmente, métodos baseados em aprendizado de máquina, como o Isolation Forest e os Autoencoders, permitem identificar padrões anômalos sutis que escapam aos testes estatísticos clássicos (NDIBE, 2025). Apesar da maturidade individual dessas técnicas, Crone e Green (2025) demonstram, em revisão sistemática recente, que cada ferramenta apresenta limitações próprias quando aplicada isoladamente: algumas exigem tamanhos de amostra mínimos, outras são sensíveis ao tipo de escala utilizada, e nenhuma oferece, por si só, um veredicto integrado sobre a confiabilidade de um conjunto de dados. Essa fragmentação impõe ao analista a necessidade de recorrer a múltiplas ferramentas distintas, interpretar resultados heterogêneos sem critério unificado e produzir relatórios manualmente — o que torna o processo caro, sujeito a erros e inacessível a profissionais sem formação estatística aprofundada. No ecossistema Python,

essa lacuna é evidente: ferramentas disponíveis no PyPI, como *benfordslaw* e *benford_py*, implementam exclusivamente a Lei de Benford, sem integração com GRIM, SPRITE, análise de entropia ou modelos de aprendizado de máquina. Não existe, até o momento, uma biblioteca que reúna essas abordagens em uma API padronizada, com pontuação de confiança unificada e geração automática de relatórios forenses. É nessa lacuna que o presente trabalho se insere, propondo o desenvolvimento da **FakeNumDetect**.

1.2 Justificativa

A realização deste trabalho justifica-se sob três perspectivas:

- **Relevância acadêmica**: o projeto integra conhecimentos de Estatística, Inteligência Artificial e Engenharia de Software em um produto publicável no PyPI, representando uma contribuição original ao ecossistema científico Python. Uma pesquisa extensiva no PyPI, GitHub e bases acadêmicas (BDTD, Google Scholar) confirmou que nenhuma biblioteca equivalente existe: ferramentas como *benfordslaw* e *benford_py* implementam apenas a Lei de Benford, sem integração com outros métodos forenses ou modelos de aprendizado de máquina.
- **Relevância prática**: a biblioteca será útil a auditores fiscais, pesquisadores, jornalistas de dados e profissionais de compliance que precisam verificar a autenticidade de conjuntos numéricos. A geração automática de relatórios forenses com pontuação de confiança torna a ferramenta acessível mesmo a usuários sem expertise estatística aprofundada.
- **Lacuna identificada**: nenhuma solução atual combina a Lei de Benford, os testes GRIM e SPRITE, análise de entropia, detecção de duplicação e modelos de aprendizado de máquina (Isolation Forest e Autoencoder) em uma única API padronizada, com interface estilo scikit-learn e suporte a relatórios HTML/PDF.

1.3 Objetivo

1.3.1 Objetivo principal

Desenvolver uma biblioteca open-source em Python, denominada FakeNumDetect, que unifique múltiplas técnicas estatísticas e de aprendizado de máquina para detecção de anomalias em dados numéricos fabricados ou manipulados, disponibilizando-a publicamente no repositório PyPI com documentação completa e validação experimental.

1.3.2 Objetivos secundários

1. Implementar os testes estatísticos de Lei de Benford (1º e 2º dígitos) com métricas de conformidade (qui-quadrado, MAD e Z-estatística);

2. Desenvolver implementações em Python dos testes GRIM e SPRITE para verificação de consistência de estatísticas reportadas;

3. Criar módulos de análise de entropia de Shannon, teste de terminação de dígitos e detecção de padrões de duplicação;

4. Integrar modelos de aprendizado de máquina — Isolation Forest e Sparse Autoencoder — para identificação de padrões sutis de fabricação;

5. Projetar um sistema de pontuação unificado (0–100) que agregue os resultados de todas as técnicas com pesos configuráveis;

6. Implementar gerador automático de relatórios forenses em HTML e PDF com visualizações gráficas;

7. Publicar a biblioteca no PyPI com cobertura de testes superior a 90%;

8. Validar a abordagem multi-técnica em datasets reais de auditoria contábil, pesquisas científicas e dados eleitorais, comparando sua eficácia com a de técnicas aplicadas individualmente.

2 Revisão Bibliográfica

Este capítulo apresenta o embasamento teórico e o estado da arte que fundamentam o desenvolvimento da FakeNumDetect. O referencial teórico cobre os principais métodos de forense numérica e as técnicas de aprendizado de máquina que serão integrados na biblioteca. Em seguida, descreve-se a metodologia de pesquisa bibliográfica adotada, aprofundam-se os conceitos técnicos centrais do trabalho e, por fim, analisam-se os trabalhos relacionados mais próximos ao problema proposto.

2.1 Referencial teórico

A detecção de anomalias em dados numéricos apoia-se em dois pilares complementares: métodos estatísticos de forense numérica, desenvolvidos ao longo de décadas de pesquisa em auditoria e integridade científica, e técnicas modernas de aprendizado de máquina, capazes de identificar padrões anômalos em espaços de alta dimensionalidade. As subseções a seguir apresentam os conceitos fundamentais de cada pilar.

2.1.1 Forense numérica e integridade de dados

A forense numérica é o campo dedicado a identificar, por meio de análise estatística e computacional, evidências de fabricação ou manipulação em conjuntos de dados numéricos. Sua aplicação abrange auditoria fiscal, verificação de integridade científica e jornalismo de dados investigativo. Conforme apontam Crone e Green (2025), a área dispõe atualmente de um conjunto robusto de ferramentas estatísticas — cada uma projetada para detectar um padrão específico de anomalia —, mas nenhuma delas oferece, isoladamente, uma avaliação abrangente da confiabilidade de um conjunto de dados. A necessidade de múltiplas perspectivas analíticas foi reforçada empiricamente por Schumm et al. (2021), que aplicaram nove técnicas distintas de detecção em artigos científicos com amostras pequenas e concluíram que a combinação de métodos produz diagnósticos mais robustos do que qualquer técnica individual. Esse achado é diretamente relevante para a proposta deste trabalho: a FakeNumDetect foi concebida precisamente para operacionalizar essa abordagem multi-técnica em uma API única.

2.1.2 Lei de Benford

A Lei de Benford, formalizada por Benford (1938) e matematicamente fundamentada por Hill (1995), estabelece que, em conjuntos de dados gerados por processos naturais e

que abrangem várias ordens de grandeza, o primeiro dígito significativo d (onde $1 \leq d \leq 9$) ocorre com probabilidade dada por:

$$P(d) = \log_{10}\left(1 + \frac{1}{d}\right) \tag{2.1}$$

Assim, o dígito 1 aparece como primeiro dígito em aproximadamente 30,1% dos casos, enquanto o dígito 9 ocorre em apenas 4,6%. Dados fabricados por humanos tendem a não seguir essa distribuição, pois as pessoas intuitivamente distribuem os dígitos de forma mais uniforme. A conformidade com a lei é verificada por meio de testes estatísticos como o qui-quadrado, o desvio absoluto médio (MAD) e a Z-estatística (NIGRINI, 2012). A lei tem sido aplicada com sucesso em domínios variados, como detecção de fraude em demonstrações financeiras (RAD et al., 2021), verificação de dados de saúde pública (BALASHOV; YAN; ZHU, 2021) e auditoria de transações em criptomoedas (VIčIč; TOšIć, 2022). No entanto, sua eficácia depende criticamente das características do conjunto de dados: Ergin e Erturan (2021) demonstram que Benford não é aplicável a dados de ciclo de despesas com distribuições restritas, o que reforça a necessidade de combinar o método com outras técnicas de detecção.

2.1.3 Teste GRIM

O teste GRIM (*Granularity-Related Inconsistency of Means*), proposto por Brown e Heathers (2017), verifica se uma média reportada é matematicamente possível dado o tamanho da amostra N e a granularidade dos itens medidos. Para dados inteiros ou de escala Likert, a média deve corresponder a um dos $N + 1$ valores representáveis com a precisão declarada. Se a média reportada não pertence a esse conjunto, há necessariamente um erro de cálculo ou fabricação. Em estudo aplicado a 260 artigos de psicologia, Brown e Heathers (2017) identificaram que aproximadamente metade dos artigos testáveis continha ao menos uma média GRIM-inconsistente, e mais de 20% apresentavam múltiplas inconsistências. A simplicidade computacional do teste — que requer apenas o valor da média, o número de itens e o tamanho da amostra — o torna especialmente adequado para implementação em uma biblioteca de uso geral como a FakeNumDetect.

2.1.4 Teste SPRITE

O SPRITE (*Sample Parameter Reconstruction via Iterative Techniques*), desenvolvido por Heathers et al. (2018), é uma extensão do GRIM que verifica a consistência mútua entre múltiplas estatísticas descritivas reportadas simultaneamente — média, desvio padrão, mínimo, máximo e tamanho da amostra. O método utiliza busca iterativa para tentar reconstruir um conjunto de dados que produza exatamente as estatísticas declaradas. A ausência de qualquer conjunto válido constitui evidência de inconsistência nos dados reportados. Schumm et al. (2023) aplicaram o SPRITE a artigos de criminologia e relataram

que, em amostras maiores, o teste apresenta limitações de especificidade. Essa observação é incorporada ao design da FakeNumDetect: o SPRITE será apresentado ao usuário com indicação explícita do intervalo de tamanho amostral em que seus resultados são mais confiáveis, e seu peso no score unificado será ajustável conforme o contexto de análise.

2.1.5 Análise de entropia de Shannon

A entropia de Shannon (SHANNON, 1948) mede o grau de aleatoriedade na distribuição de símbolos de um conjunto. Aplicada à distribuição de dígitos (0–9) em um conjunto de dados numéricos, é calculada como:

$$H = -\sum_{x=0}^{9} p(x) \cdot \log_2 p(x) \tag{2.2}$$

onde $p(x)$ é a frequência relativa de cada dígito. Dados genuinamente aleatórios tendem a apresentar entropia próxima do máximo teórico ($\log_2 10 \approx 3{,}32$ bits), enquanto dados fabricados por humanos frequentemente exibem entropia anormalmente baixa — pela tendência de repetir certos dígitos — ou artificialmente elevada — por tentativas deliberadas de simular aleatoriedade (MOSIMANN; WISEMAN; EDELMAN, 1995).

2.1.6 Isolation Forest

O Isolation Forest, proposto por Liu, Ting e Zhou (2008), é um algoritmo de detecção de anomalias baseado na premissa de que pontos anômalos são mais fáceis de isolar do que pontos normais em uma árvore de decisão aleatória. O algoritmo constrói um conjunto de árvores (*forest*) que particionam aleatoriamente o espaço de atributos e mede o comprimento médio do caminho necessário para isolar cada observação: pontos que requerem poucas partições para serem isolados recebem scores de anomalia elevados. Na FakeNumDetect, o Isolation Forest será aplicado sobre o vetor de *features* estatísticas extraídas de cada análise — distribuição de dígitos, entropia, assimetria, curtose — para capturar padrões sutis de fabricação que os testes individuais não detectam. Aguilar et al. (2023) demonstram que o Isolation Forest mantém desempenho competitivo frente a alternativas como One-Class SVM e Gaussian Mixture, com a vantagem adicional da escalabilidade para grandes volumes de dados.

2.1.7 Autoencoders para detecção de anomalias

Um autoencoder é uma rede neural treinada para reconstruir sua própria entrada por meio de uma representação compacta em camada latente (NG, 2011). Quando treinado exclusivamente com dados considerados genuínos, o modelo aprende a representar os padrões normais com baixo erro de reconstrução. Dados anômalos — como conjuntos fabricados — produzem erros de reconstrução elevados, pois diferem dos padrões aprendidos

durante o treinamento. Ndibe (2025) demonstra que autoencoders integrados a fluxos de forense digital superam abordagens puramente baseadas em regras na detecção de ameaças com baixo volume de sinal. Na FakeNumDetect, um Sparse Autoencoder será empregado como complemento ao Isolation Forest, oferecendo uma segunda perspectiva de ML sobre o mesmo conjunto de *features* estatísticas.

2.2 Metodologia da pesquisa bibliográfica

A pesquisa bibliográfica foi conduzida nas bases Google Scholar e ResearchRabbit. As buscas utilizaram os seguintes termos em inglês, de forma isolada e combinada usando técnica de funil: *data fabrication detection*, *numerical anomaly detection*, *Benford's law fraud*, *GRIM test*, *SPRITE test*, *forensic data analytics*, *isolation forest anomaly*, *autoencoder anomaly detection* e *scientific misconduct statistics*. Foram adotados os seguintes critérios de inclusão: (i) publicações em inglês; (ii) período de 2021 a 2026 para os artigos principais de trabalhos relacionados; (iii) relevância temática direta com detecção de anomalias em dados numéricos tabulares, forense numérica ou aprendizado de máquina aplicado a integridade de dados. Foram excluídos trabalhos que aplicam Benford exclusivamente a domínios não tabulares (imagens, texto gerado por IA), artigos anteriores a 2021 — salvo referências seminais insubstituíveis — e publicações sem revisão por pares. O processo resultou em 41 artigos triados, dos quais 3 foram selecionados como trabalhos relacionados principais, 8 compõem o referencial teórico e os demais foram excluídos por não atenderem aos critérios de inclusão ou por abordarem temas diferentes não relacionados.

2.3 Fundamentos técnicos

Esta seção detalha os conceitos técnicos diretamente empregados na implementação da FakeNumDetect, complementando o referencial teórico com aspectos de engenharia de software e design de API.

2.3.1 Design de API estilo scikit-learn

O scikit-learn (PEDREGOSA et al., 2011) estabeleceu na comunidade Python um padrão de API para aprendizado de máquina baseado em três operações principais: `fit()` para ajuste do modelo aos dados, `predict()` ou `transform()` para aplicação do modelo, e `score()` para avaliação de desempenho. A consistência desse padrão permite que ferramentas distintas sejam intercambiáveis em pipelines de análise. A FakeNumDetect adotará esse mesmo padrão — com a interface `fit()`/`detect()`/`report()` — para garantir familiaridade imediata a usuários do ecossistema Python científico.

2.3.2 Score de confiança unificado

O score de confiança unificado (0–100) da FakeNumDetect será calculado como uma média ponderada dos resultados individuais de cada técnica, onde os pesos refletem a capacidade discriminativa de cada método em função das características do conjunto de dados analisado. Cada técnica produz um valor normalizado de suspeição, que é então combinado por uma função de agregação configurável pelo usuário. Esse design permite que analistas com conhecimento do domínio ajustem os pesos conforme a natureza dos dados — por exemplo, aumentando o peso do GRIM em datasets de pesquisas com escalas Likert ou o peso de Benford em dados financeiros de larga escala.

2.3.3 Geração de relatórios forenses

Os relatórios forenses da FakeNumDetect serão gerados em HTML e PDF a partir de templates Jinja2, incorporando visualizações produzidas com Matplotlib e Plotly. Cada relatório incluirá o score de confiança unificado com interpretação textual, os resultados detalhados de cada técnica com seus respectivos p-valores e métricas de desvio, gráficos de distribuição de dígitos comparados à distribuição esperada por Benford, e uma seção de conclusão com indicação do nível de suspeita (baixo, moderado, alto, muito alto).

2.4 Trabalhos relacionados

Os três trabalhos apresentados a seguir foram selecionados por enfrentarem diretamente o mesmo problema central da FakeNumDetect: a detecção automatizada de anomalias em dados numéricos por meio da combinação de múltiplas técnicas analíticas. Cada um deles avança em relação ao uso isolado de uma única técnica, mas deixa lacunas específicas que o presente trabalho se propõe a superar.

2.4.1 Trabalho 1: *Tools of the data detective* (Crone e Green, 2025)

Crone e Green (2025) enfrentaram o problema de avaliar sistematicamente o conjunto de ferramentas estatísticas disponíveis para detecção de fabricação de dados em pesquisas de psicologia. O trabalho realizou uma revisão abrangente de métodos como GRIM, SPRITE, análise de distribuição de dígitos, testes de variância e medidas de tamanho de efeito, avaliando as forças e limitações de cada ferramenta tanto na análise de dados brutos quanto de estatísticas resumidas reportadas em artigos. Os resultados demonstram que os testes de variância e as medidas de tamanho de efeito apresentam desempenho de bom a excelente na detecção de dados fabricados a partir de dados brutos, enquanto a análise de dígitos isolada performa próximo ao acaso — o que contradiz seu uso difundido como ferramenta diagnóstica única. Os autores concluem que nenhuma ferramenta individual

é suficiente e que a combinação de múltiplos métodos é necessária para diagnósticos confiáveis. A limitação central do trabalho é seu caráter exclusivamente descritivo e de revisão: Crone e Green (2025) avaliam ferramentas existentes, mas não propõem nenhuma implementação integrada, API, score unificado ou mecanismo de geração de relatórios. O analista que desejar aplicar os métodos combinados ainda precisa recorrer a implementações fragmentadas. O presente trabalho avança nesse ponto ao traduzir exatamente essa necessidade — identificada e documentada por Crone e Green — em uma biblioteca Python funcional, publicável e de uso imediato.

2.4.2 Trabalho 2: *Integrating Machine Learning with Digital Forensics* (Ndibe, 2025)

Ndibe (2025) abordou o problema de limitação das abordagens de forense digital tradicionais diante da crescente complexidade e volume de evidências digitais. O trabalho investigou a integração de modelos de aprendizado de máquina — especificamente autoencoders, random forests e redes LSTM — em fluxos de trabalho de forense computacional, com foco em detecção de anomalias em redes, ambientes de nuvem e sistemas industriais. Os resultados mostram que modelos de ML treinados para aprender o comportamento baseline de sistemas legítimos são capazes de detectar desvios com desempenho superior às abordagens baseadas em regras, especialmente em cenários com ataques de baixo sinal e ameaças não catalogadas previamente. O trabalho enfatiza ainda a importância de interpretabilidade e robustez adversarial para a adoção de ML em contextos forenses de alto risco. A limitação principal em relação ao escopo da FakeNumDetect é de domínio: Ndibe (2025) concentra-se em forense de redes e sistemas cibernéticos, sem tratar de dados numéricos tabulares, forense científica ou auditoria contábil. Não há integração com testes estatísticos clássicos como Benford, GRIM ou SPRITE, nem geração de relatórios forenses padronizados. O presente trabalho se diferencia ao aplicar a mesma lógica de integração de ML em um domínio distinto — dados numéricos tabulares —, combinando-a com o arsenal de testes estatísticos de forense numérica que Ndibe (2025) não contempla.

2.4.3 Trabalho 3: *Integrating Data Mining Techniques for Fraud Detection* (Sushkov et al., 2023)

Sushkov et al. (2023) enfrentaram o problema das limitações práticas da Lei de Benford quando aplicada isoladamente em processos de controle financeiro. Os autores propuseram uma metodologia que combina os testes estatísticos de Benford — incluindo testes de primeiro dígito, segundo dígito e combinados — com algoritmos de clustering para identificar classes de transações suspeitas que o teste de Benford por si só não consegue isolar com precisão suficiente. A validação foi conduzida em dados reais de auditoria de uma empresa de construção de estradas, e os resultados demonstram que

a abordagem combinada reduziu significativamente o volume de transações que exigem revisão manual, identificando um subconjunto restrito de transações altamente suspeitas em meio a um grande volume de registros. Os autores concluem que a integração de Benford com técnicas de mineração de dados é essencial para escalar a auditoria forense a contextos organizacionais complexos. As limitações do trabalho em relação à FakeNumDetect são três. Primeiro, a integração proposta é restrita a Benford com clustering, sem incorporação de GRIM, SPRITE, análise de entropia, teste de terminação ou modelos de ML como Isolation Forest e Autoencoder. Segundo, a metodologia é descrita como um processo manual de análise, sem implementação em uma biblioteca reutilizável com API padronizada. Terceiro, não há sistema de pontuação unificado nem geração automática de relatórios. O presente trabalho avança em todas essas dimensões: amplia o conjunto de técnicas integradas, encapsula a metodologia em uma API estilo scikit-learn e automatiza tanto o scoring quanto a geração de relatórios forenses.

A Tabela 1 sintetiza as principais dimensões de comparação entre os trabalhos relacionados e a FakeNumDetect.

Tabela 1 – Comparação entre trabalhos relacionados e a FakeNumDetect

Critério	**Crone e Green (2025)**	**Ndibe (2025)**	**Sushkov et al. (2023)**	**FakeNumDetect (este trabalho)**
Problema central	Avaliação de ferramentas de detecção	Forense digital em redes/sistemas	Limitação de Benford em auditoria	Ausência de API multi-técnica unificada
Técnicas estatísticas	GRIM, SPRITE, dígitos, variância	Não aplicável (domínio de redes)	Lei de Benford (1º e 2º dígito)	Benford, GRIM, SPRITE, entropia, terminação
Aprendizado de máquina	Não aplicável	Autoencoder, RF, LSTM	Clustering (K-Means)	Isolation Forest + Sparse Autoencoder
Score unificado	Não	Não	Não	Sim (0–100, pesos configuráveis)
API reutilizável	Não (revisão teórica)	Não (estudo de caso)	Não (metodologia manual)	Sim (PyPI, estilo scikit-learn)
Relatórios automáticos	Não	Não	Não	Sim (HTML e PDF com visualizações)
Validação empírica	Revisão sistemática	Estudos de caso em redes	Auditoria em empresa real	Datasets reais de 3 domínios distintos
Limitação principal	Não propõe implementação	Domínio restrito a redes	Técnica única, sem API	—

Fonte: Elaborado pela autora (2026)

3 Desenvolvimento

Neste capítulo, apresenta-se a solução proposta para o problema identificado, bem como o cronograma de trabalho previsto para o desenvolvimento completo do projeto no TCC2.

3.1 Solução proposta

Descreva detalhadamente a solução que será desenvolvida para resolver o problema apresentado na introdução. A descrição deve incluir:

- **Visão geral da solução**: o que será desenvolvido (sistema, aplicativo, modelo, ferramenta, etc.);
- **Arquitetura**: como os componentes da solução se organizam e interagem;
- **Tecnologias e ferramentas**: quais linguagens, frameworks, bibliotecas, bancos de dados e ambientes serão utilizados, e por que foram escolhidos;
- **Funcionalidades principais**: o que a solução fará para atender aos objetivos;
- **Metodologia de desenvolvimento**: qual abordagem será utilizada (ágil, cascata, prototipação, etc.);
- **Critérios de validação**: como será medido o sucesso da solução (métricas, testes, avaliação com usuários, etc.).

Utilize diagramas, fluxogramas ou mockups para ilustrar a solução. Exemplo de descrição de arquitetura:

A solução proposta consiste em uma aplicação web composta por três camadas: front-end desenvolvido em React.js, back-end em Node.js com Express, e banco de dados PostgreSQL. A comunicação entre front-end e back-end será feita por meio de uma API REST.

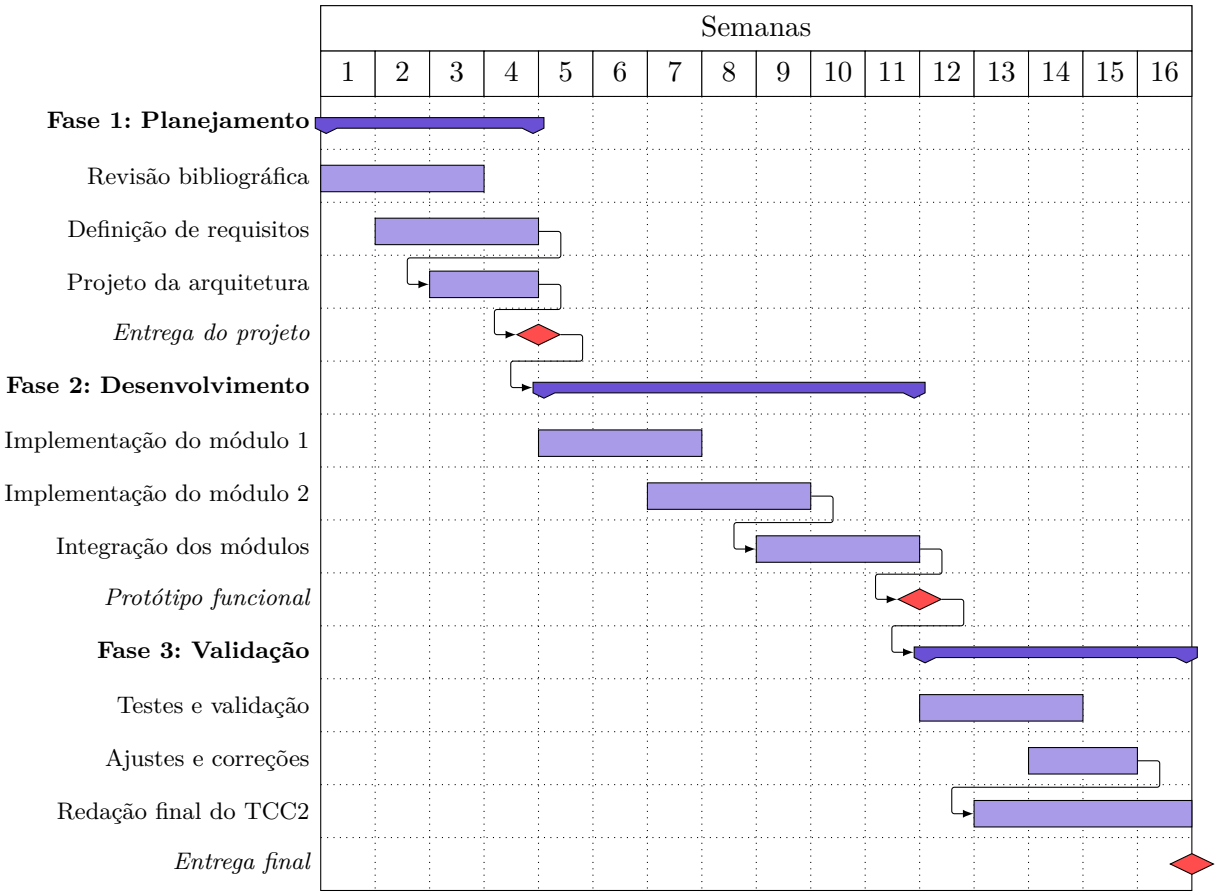
3.2 Cronograma de trabalho

O cronograma a seguir apresenta, no formato de diagrama de Gantt, as atividades previstas para o desenvolvimento do TCC2, distribuídas ao longo das semanas do semestre.

Instruções para personalização do cronograma:

- Ajuste o número de semanas conforme o calendário acadêmico do semestre;
- Modifique as atividades de acordo com as etapas do seu projeto;

Figura 1 – Cronograma de atividades – Diagrama de Gantt



Fonte: Elaborado pelo autor (2026)

- Os marcos (*milestones*) indicam entregas importantes — adapte-os às datas de avaliação;
- As setas entre atividades indicam dependências — uma atividade só inicia após a conclusão da anterior;
- Utilize o diagrama como ferramenta de acompanhamento: atualize-o a cada reunião com o orientador.

Referências

AGUILAR, D. L. et al. Towards an interpretable autoencoder: A decision-tree-based autoencoder and its application in anomaly detection. *IEEE Transactions on Dependable and Secure Computing*, v. 20, n. 3, p. 2021–2038, 2023. 9

BALASHOV, V. S.; YAN, Y.; ZHU, X. Using the Newcomb–Benford law to study the association between a country's COVID-19 reporting accuracy and its development. *Scientific Reports*, v. 11, n. 23406, 2021. 8

BENFORD, F. The law of anomalous numbers. *Proceedings of the American Philosophical Society*, v. 78, n. 4, p. 551–572, 1938. 4, 7

BROWN, N. J. L.; HEATHERS, J. A. J. The GRIM test: A simple technique detects numerous anomalies in the reporting of results in psychology. *Social Psychological and Personality Science*, v. 8, n. 4, p. 363–369, 2017. 4, 8

CRONE, G.; GREEN, C. D. Tools of the data detective: A review of statistical methods to detect data and result anomalies in psychology. *Theory & Psychology*, 2025. 4, 7, 11, 12

ERGIN, E.; ERTURAN İlkay E. Is Benford's law effective in fraud detection for expense cycle? *Marmara Üniversitesi İktisadi ve İdari Bilimler Dergisi*, v. 43, n. 2, p. 186–207, 2021. 8

HEATHERS, J. A. J. et al. Recovering data from summary statistics: Sample parameter reconstruction via iterative techniques (SPRITE). In: *PeerJ Preprints*. [S.l.: s.n.], 2018. 4, 8

HILL, T. P. A statistical derivation of the significant-digit law. *Statistical Science*, v. 10, n. 4, p. 354–363, 1995. 7

LIU, F. T.; TING, K. M.; ZHOU, Z.-H. Isolation forest. In: *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*. Pisa: IEEE, 2008. p. 413–422. 9

MOSIMANN, J. E.; WISEMAN, C. V.; EDELMAN, R. E. Data fabrication: Can people generate random digits? *Accountability in Research*, v. 4, n. 1, p. 31–55, 1995. 9

NDIBE, O. S. Integrating machine learning with digital forensics to enhance anomaly detection and mitigation strategies. *International Journal of Research Publication and Reviews*, v. 6, n. 5, 2025. 4, 10, 12

NG, A. *Sparse autoencoder*. 2011. CS294A Lecture Notes. Stanford University, Stanford. Disponível em: <https://web.stanford.edu/class/cs294a/sparseAutoencoder.pdf>. 9

NIGRINI, M. J. *Benford's Law: Applications for Forensic Accounting, Auditing, and Fraud Detection*. Hoboken: John Wiley & Sons, 2012. 8

PEDREGOSA, F. et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011. 10

RAD, M. S. et al. Predictability of financial statements fraud-risk using Benford's law. *Cogent Economics & Finance*, v. 9, n. 1, 2021. 8

SCHUMM, W. R. et al. Nine ways to detect possible scientific misconduct in research with small (N < 200) samples. *Psychology Research and Applications*, v. 3, n. 2, p. 1–18, 2021. 7

SCHUMM, W. R. et al. Research anomalies in criminology: How serious? how extensive over time? and who was responsible? *Accountability in Research*, 2023. 8

SHANNON, C. E. A mathematical theory of communication. *Bell System Technical Journal*, v. 27, n. 3, p. 379–423, 1948. 9

SIMONSOHN, U. Just post it: The lesson from two cases of fabricated data detected by statistics alone. *Psychological Science*, v. 24, n. 10, p. 1875–1888, 2013. 4

SUSHKOV, V. M. et al. Integrating data mining techniques for fraud detection in financial control processes. *International Journal of Technology*, v. 14, n. 8, p. 1718–1728, 2023. 12

USTINOVA, Y. The impact of accounting fraud which leads to financial crimes: accounting fraud. *SHS Web of Conferences*, 2022. 4

VIčIč, J.; TOšIć, A. Application of Benford's law on cryptocurrencies. *Journal of Theoretical and Applied Electronic Commerce Research*, v. 17, n. 1, p. 313–328, 2022. 8